

Contents

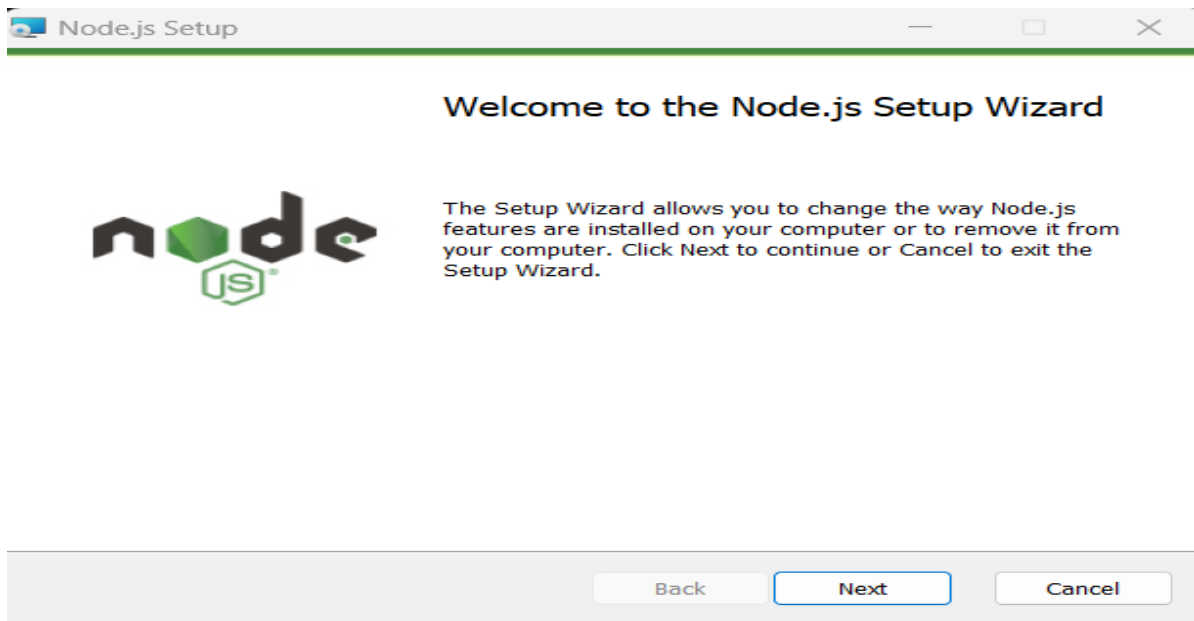
Download Node.js:.....	2
Verify Installation:.....	2
Environment Variables:.....	2
Project Dependencies and Details	4
Install Project Dependencies:	4
Dependencies and its uses.....	5
Modules and Structures	5
1. Setting Up the Web Server (with Express):	5
2. Establishing Data Base Connection:	5
3. .env	6
4. Configuring Routes and Middleware:	7
5. Router Folder	8
6. Readme.md	8
Running and Testing	8
Run the Application.....	8
Final Check.....	9

Node.js Installation and Setup Guide

Download Node.js:

- Go to the official Node.js website: <https://nodejs.org/>
- Download the **LTS version** (Long-Term Support) for stability, unless you need the latest features provided by the **Current version**.

Select the installer appropriate for your operating system (Windows, macOS, or Linux).



Verify Installation:

After installation, open a terminal or command prompt and run the following commands to check if Node.js and npm (Node Package Manager) are correctly installed:

- **Check Node.js version:**

```
node -v
```

This should output the installed version of Node.js (e.g., v18.17.1).

- **Check npm version:**

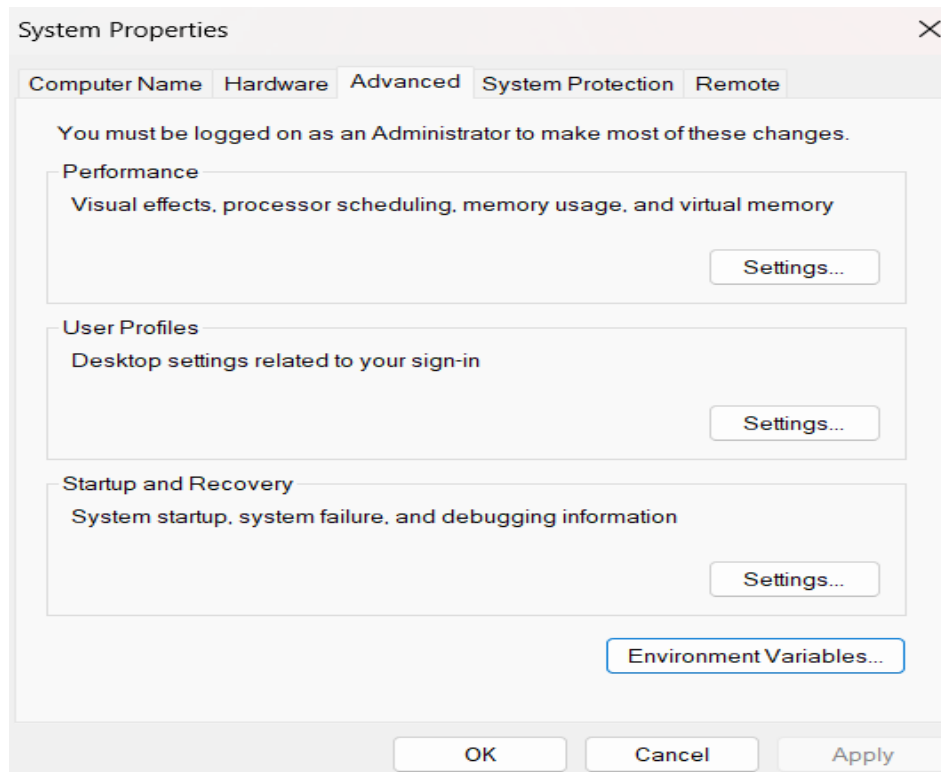
```
npm -v
```

This will output the installed version of npm (e.g., 8.19.2).

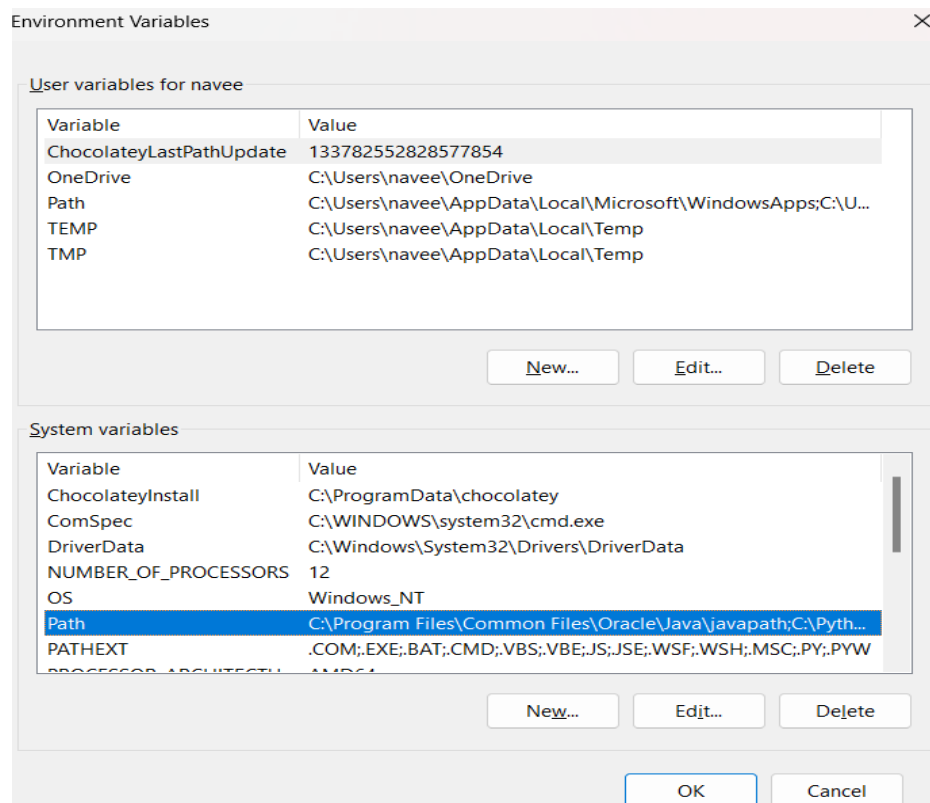
Environment Variables:

1. Search for "Environment Variables" in the Start menu.

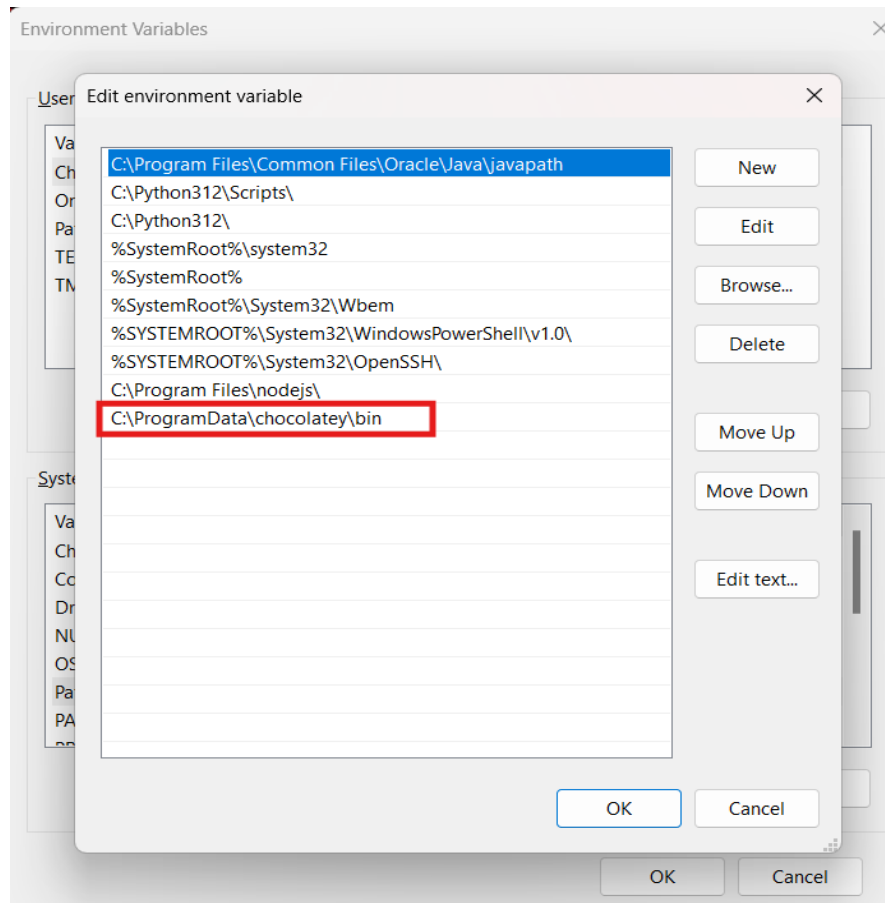
2. Open "Edit the system environment variables".
3. Click Environment Variables.



4. Under System Variables, find and select Path, then click Edit



5. Add C:\Program Files\nodejs\ (or wherever Node.js is installed) to the list.



6. Click OK and restart your terminal

Project Dependencies and Details

Install Project Dependencies:

Next, you can install the necessary dependencies for your Node.js backend project using terminal.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\ navee\OneDrive\Desktop\JSw> npm install express dotenv cors mysql2 nodemailer multer nodemon
```

By using the below code we can install the required dependencies

```
npm install express dotenv cors mysql2 nodemailer multer nodemon
```

Dependencies and its uses

- **Express** (web framework)
- **dotenv** (to manage environment variables)
- **cors** (for handling cross-origin requests)
- **mysql2** (for MySQL integration)
- **nodemailer** (to send mail)
- **multer** (to handle image/pdf)
- **nodemon** (to auto restart server while saving changes)

```
"cors": "^2.8.5",  
"dotenv": "^16.4.5",  
"express": "^4.19.2",  
"http": "^0.0.1-security",  
"multer": "^1.4.5-lts.1",  
"mysql": "^2.18.1",  
"mysql2": "^3.11.2",  
"nodemailer": "^6.9.16",  
"nodemon": "^3.1.7"
```

Modules and Structures

1. Setting Up the Web Server (with Express):

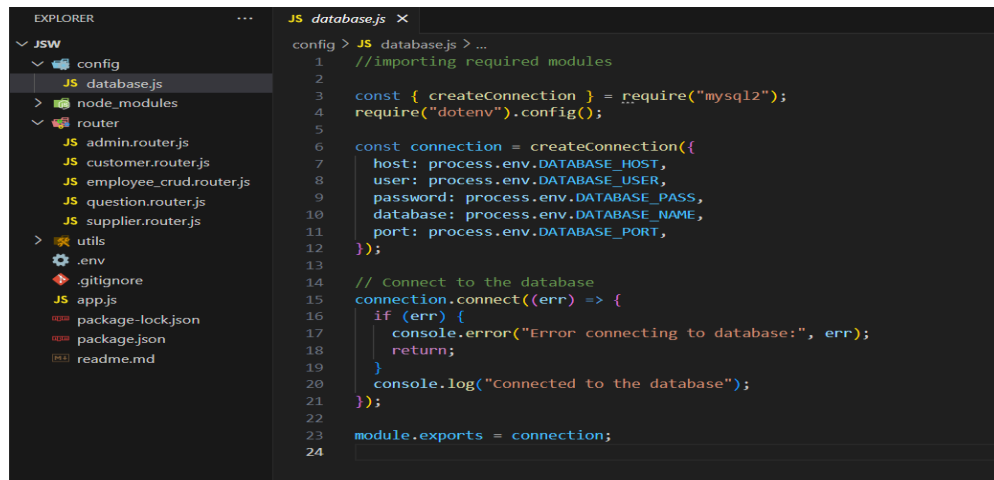
In most modern Node.js applications, `app.js` is used to set up an Express server. Express is a lightweight web framework for building APIs and handling HTTP requests.

In dependencies we use express as module by applying it in the terminal

```
npm install express
```

2. Establishing Data Base Connection:

Here we initialize Data Base connection using CONFIG folder -> `database.js`.



```
1 //importing required modules
2
3 const { createConnection } = require("mysql2");
4 require("dotenv").config();
5
6 const connection = createConnection({
7   host: process.env.DATABASE_HOST,
8   user: process.env.DATABASE_USER,
9   password: process.env.DATABASE_PASS,
10  database: process.env.DATABASE_NAME,
11  port: process.env.DATABASE_PORT,
12 });
13
14 // Connect to the database
15 connection.connect((err) => {
16   if (err) {
17     console.error("Error connecting to database:", err);
18     return;
19   }
20   console.log("Connected to the database");
21 });
22
23 module.exports = connection;
24
```

3. .env

While Establishing Data Base connection we use process.env.DATABASE_Details

For security reasons and to manage sensitive data like API keys or database credentials, you should store environment variables in a .env file.

Database configuration

DATABASE_HOST='bp9d6oj7d523sjcxnm0r-mysql.services.clever-cloud.com'

DATABASE_PASS='gqgbLisZzRaLPNonSNK2'

DATABASE_NAME='bp9d6oj7d523sjcxnm0r'

DATABASE_PORT=3306

DATABASE_USER='ufhbsidkk15bsvev'

App configuration

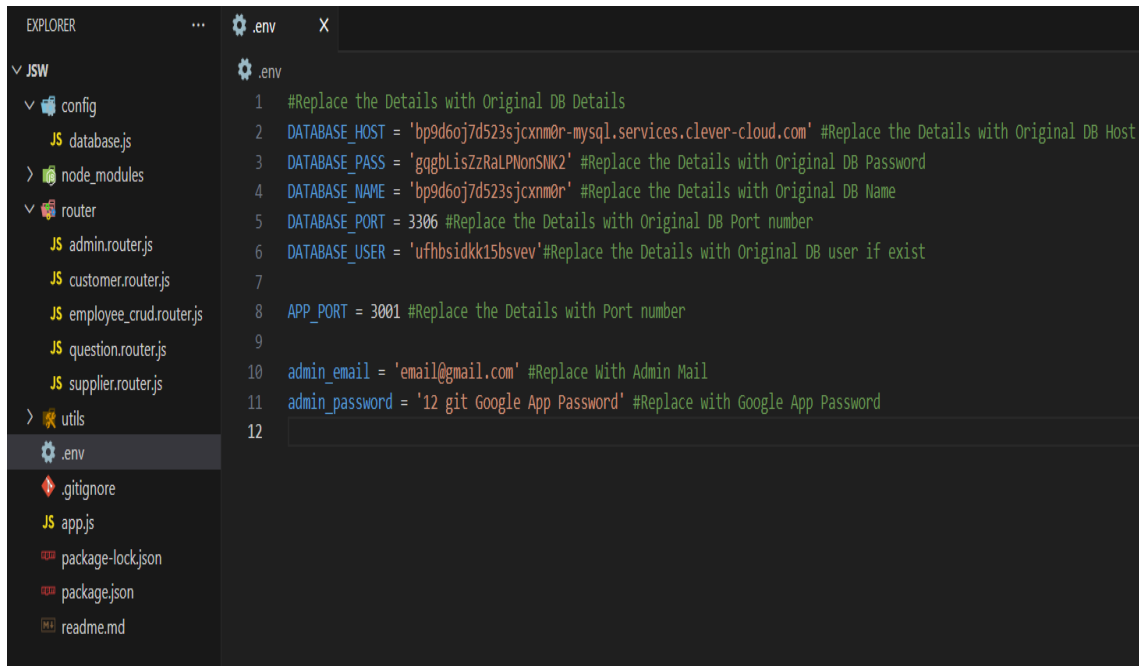
APP_PORT=3001

Admin email and password

ADMIN_EMAIL='mailid@gmail.com' //Admin mail id to be replaced

ADMIN_PASSWORD='App password manager' //12 digit code to be replaced

In this place we use .env File to safeguard data

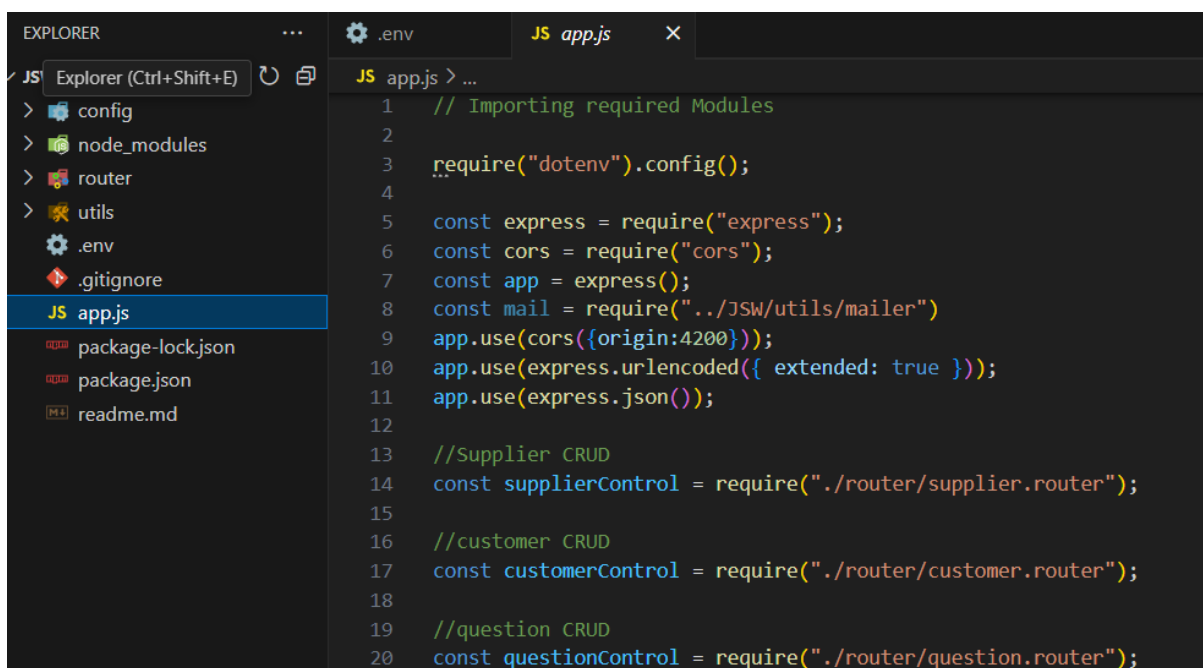


The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left and the .env file open in the editor. The Explorer sidebar shows a project structure with folders like JSW, config, node_modules, router, and utils. The .env file contains the following configuration:

```
1 #Replace the Details with Original DB Details
2 DATABASE_HOST = 'bp9d6oj7d523sjcxnm0r-mysql.services.clever-cloud.com' #Replace the Details with Original DB Host
3 DATABASE_PASS = 'ggqblisZzRaLPNonSNK2' #Replace the Details with Original DB Password
4 DATABASE_NAME = 'bp9d6oj7d523sjcxnm0r' #Replace the Details with Original DB Name
5 DATABASE_PORT = 3306 #Replace the Details with Original DB Port number
6 DATABASE_USER = 'urfbsidkk15bsvev' #Replace the Details with Original DB user if exist
7
8 APP_PORT = 3001 #Replace the Details with Port number
9
10 admin_email = 'email@gmail.com' #Replace With Admin Mail
11 admin_password = '12 git Google App Password' #Replace with Google App Password
12
```

4. Configuring Routes and Middleware:

In app.js, you set up routes (HTTP request handlers like GET, POST, etc.) and middleware (functions that run during the request-response cycle, such as cors for handling cross-origin requests or body-parser for parsing incoming request bodies)



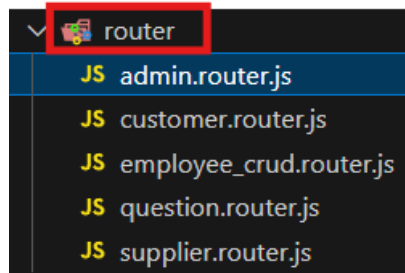
The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left and the app.js file open in the editor. The Explorer sidebar shows the project structure with folders like JSW, config, node_modules, router, and utils. The app.js file contains the following configuration:

```
1 // Importing required Modules
2
3 require("dotenv").config();
4
5 const express = require("express");
6 const cors = require("cors");
7 const app = express();
8 const mail = require("../JSW/utils/mailer");
9 app.use(cors({origin:4200}));
10 app.use(express.urlencoded({ extended: true }));
11 app.use(express.json());
12
13 //Supplier CRUD
14 const supplierControl = require("./router/supplier.router");
15
16 //customer CRUD
17 const customerControl = require("./router/customer.router");
18
19 //question CRUD
20 const questionControl = require("./router/question.router");
```

Finally, you use `app.listen()` to start the server, which listens for incoming HTTP requests on a specified port.

5.Router Folder

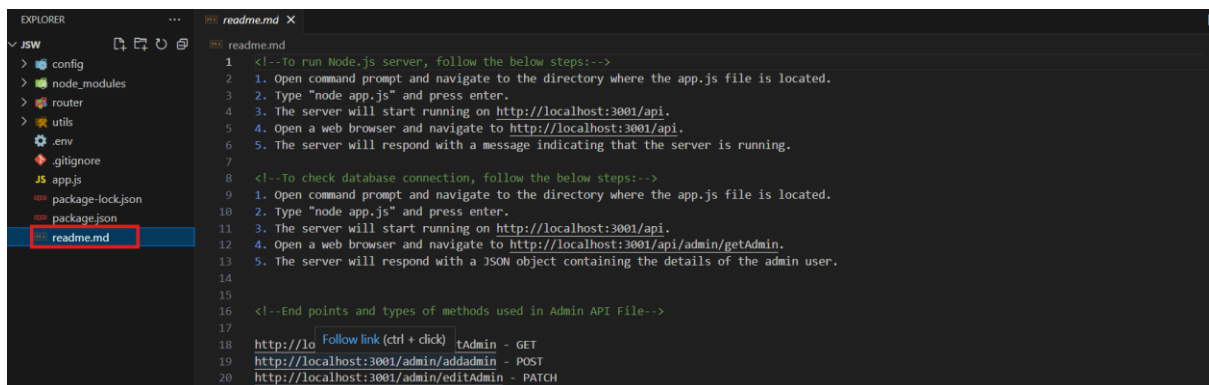
In this Router folder we consist of File which are responsible for manipulation of data from Data Base



Utils folder consists of `mailer.js` file which consists all API for mailing as notification

6. Readme.md

This File provides a comprehensive overview of the Node.js API server setup, available endpoints, and other important project details



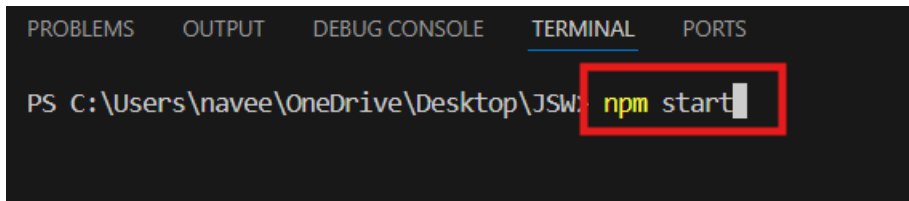
Running and Testing

Run the Application

Once you've set up your `index.js` file, you can start the server using the following command:

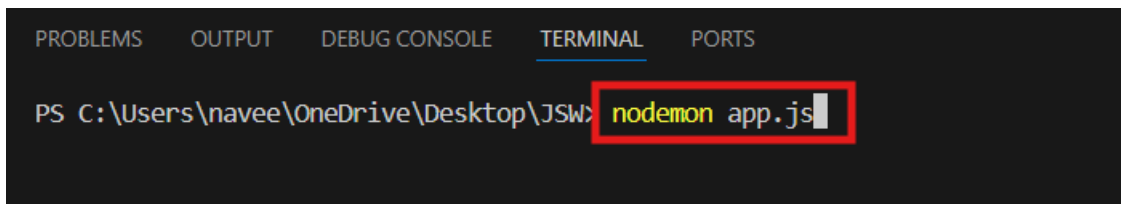
- **Start the server:**

npm start

A screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The command prompt shows 'PS C:\Users\ navee\OneDrive\Desktop\JSw>' followed by 'npm start' which is highlighted with a red rectangular box.

- **For development mode with automatic restarts (using nodemon):**

nodemon app.js

A screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The command prompt shows 'PS C:\Users\ navee\OneDrive\Desktop\JSw>' followed by 'nodemon app.js' which is highlighted with a red rectangular box.

You should see output like:

Server is running on port 3001

Now, you can open a web browser and visit <http://localhost:3001> to see your backend in action. You should get the "API is working" message.

Final Check

Make sure everything is set up correctly:

1. Node.js and npm are installed (node -v and npm -v).
 2. Your index.js file runs without errors (npm start or npm run dev).
 3. Environment variables are correctly loaded.
 4. Dependencies are properly installed and functional.
-